2026 FINTECHATHON · SUBMISSION TEMPLATE

International AI Track | Technical Document

Required material for the International AI Track. Please complete this document in English.

Team name	Team FarmLink (American International University-Bangladesh)
Competition track	International AI Track · Topic E: SME Finance Copilot
Submission date	20 October 2026

FarmLink AI: an SME finance copilot for agricultural cooperatives

Live web app: https://tanbin-cs.github.io/Farmlink-AI/

Source code: https://github.com/Tanbin-CS/Farmlink-AI

Live API health: https://farmlink-api-kfvi.onrender.com/v1/health

Execution evidence: `docs/evidence/` in the repository (unedited exports from the app)

Scope statement. FarmLink AI is a competition prototype. It runs against a sandbox: no real loan, payment, lender submission, identity check or customer data is involved. All demo businesses and lender products are fictional and labelled as such in the app.

01 Project Overview

- Problem, target users and selected topic

  Selected topic: Topic E, SME Finance Copilot, under the competition theme "AI as a Financial Participant".

  Target users: agricultural cooperatives and small agribusinesses in Bangladesh, and the managers or bookkeepers who plan their cash. A typical user is a rice cooperative that pays for seed, fertilizer and labour weeks before it sells the harvest.

  The problem has three parts:

  - **A predictable gap nobody measures.** Costs come before harvest income. The shortfall is seasonal and foreseeable, but small businesses rarely project cash day by day, so they do not know when cash runs out or by how much.
  - **Financing is compared by headline rate.** Whether the money arrives before the gap opens, and whether repayment falls after the harvest, matter more than the rate but are rarely checked.
  - **Unsafe automation.** An AI agent that can move money, approve a loan or invent numbers is not acceptable in finance. An agent must be checked, logged and approved by a human.

  FarmLink AI turns a cooperative's records into a dated cash forecast, finds the exact funding gap, compares financing options sized to that gap, stress-tests repayment, and prepares a sandbox application that only a human can submit.

- Core value and innovation
  - **A dated, exact cash gap.** Daily closing cash gives the first negative day, the depth of the gap and the recovery date. In the fictional demo cooperative, cash runs out in 7 days (27 Oct 2026) and the funding gap is BDT 425,000.
  - **Financing sized to the need, with reasons.** For each product the engine finds the smallest principal whose net proceeds cover the gap, then checks disbursement timing, maturity and cash after repayment. Every option is classified as fits, does not fit or needs information, with a reason code such as DELAYED_DISBURSEMENT, SHORTAGE_AFTER_REPAYMENT or MISSING_TERMS. An unknown fee is never treated as zero.
  - **Repayment stress test.** The drafted loan is held fixed and the season is replayed with four adverse scenarios. In the demo, the lowest-cost option survives 0 of 4 scenarios if the cooperative borrows only the gap, and 2 of 4 with the "Gap + buffer" target. The cheapest-looking option is not automatically the safest, and the user sees this before confirming.
  - **AI as a financial participant, under guardrails.** Gemini plans and proposes tool calls. Our code validates and executes them, the agent stops when an action needs approval, and only the human's click submits. Every model turn, tool proposal and policy decision appears in a run timeline and in the exportable evidence bundle.
  - **One deterministic engine for browser and server.** The same TypeScript engine with integer money runs in the browser and in the API, so guided and live modes produce identical numbers (checked by a parity test). The model never does arithmetic.
  - **Honest by design.** Two clearly labelled modes: "Guided sandbox — no live AI" (works with no server and no AI) and "Live AI + sandbox tools". Fictional data is labelled, and results say "fits the stated sandbox rules", never "approved".
  - **Usable by international reviewers and local users.** BDT, USD or EUR workspaces; approximate display conversion with editable reference rates; international or lakh digit grouping; a mobile layout; glossary tooltips; a guided five-step journey; and a command palette.

02 System Architecture

- Architecture diagram and component responsibilities

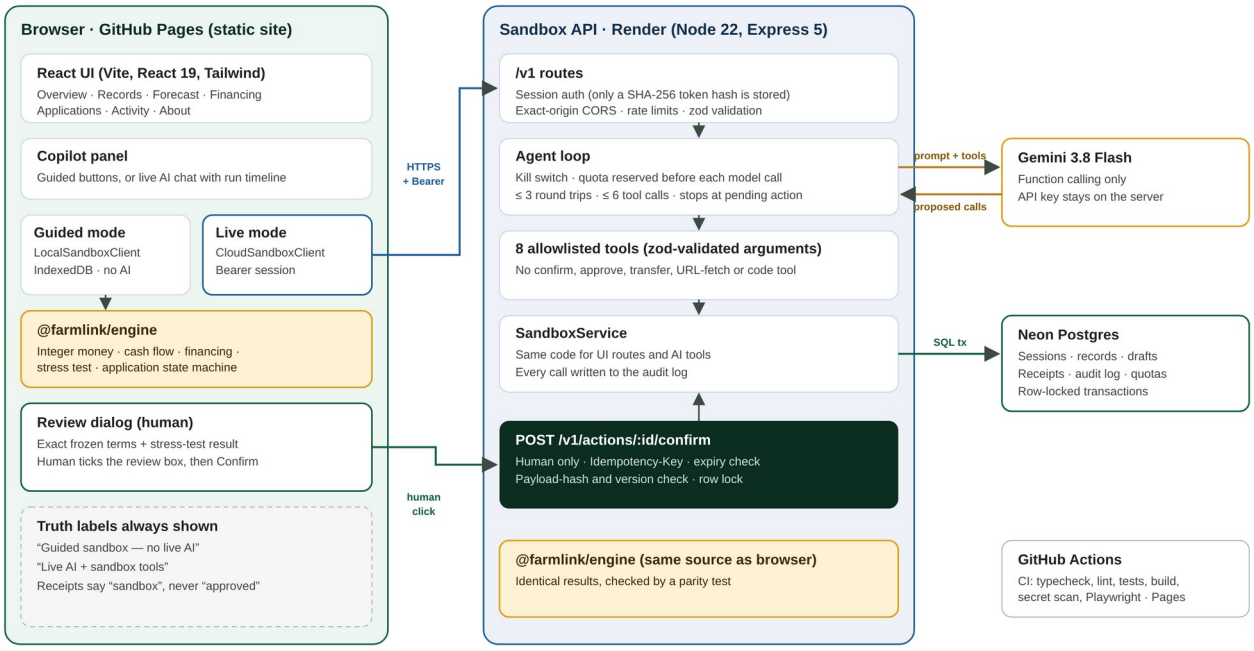


Figure 1. FarmLink AI components. The browser and the API import the same engine; only the human review dialog can reach the confirmation endpoint.

Component	Technology	Responsibility
Web app	React 19, Vite 7, TypeScript, Tailwind CSS 4, Radix UI, Recharts; GitHub Pages	Records, forecast, financing, applications and activity screens; copilot panel; review dialog. Static site with no secrets.
Guided client	LocalSandboxClient, IndexedDB	Runs the whole workflow in the browser, with no server and no AI. Explicit buttons call the same tool functions.
Live client	CloudSandboxClient	Calls the API with a bearer session held in sessionStorage. Live mode is enabled only after a successful health check.
Shared engine	@farmlink/engine (TypeScript, BigInt, zod)	Money and date types, schemas, cash-flow projection, financing comparison, stress test, application state machine, payload hashing, CSV validation, evidence bundle.
Sandbox API	Node 22, Express 5, zod, helmet, express-rate-limit; Render (Singapore)	/v1 routes, anonymous sessions, ownership, quotas, agent loop, human-only confirmation endpoint.
Agent loop and tools	backend/src/agent	Calls the model, checks each proposed call against the allowlist and schema, executes it through SandboxService, stops at a pending action.
Storage	Neon Postgres (in-memory store for tests)	Session-scoped documents, row-locked transactions, atomic quota counters, receipts and audit log.

Component	Technology	Responsibility
CI/CD	GitHub Actions	Typecheck, lint, all tests, build, secret scan and Playwright on every push; GitHub Pages deployment.

Request flow (live mode). The browser sends the user's message to `POST /v1/agent/runs` with its bearer session. The API checks the kill switch and quotas, asks Gemini for a plan, and executes each allowed tool call through SandboxService, the same service the UI routes use. Results come from the engine and the database. When an action needs approval, the run stops and the browser shows the review dialog. The human's Confirm click calls `POST /v1/actions/{id}/confirm`, which is not reachable from the model.

- Models, tools, data sources and external services

Item	Details
Model	Gemini 3.8 Flash (`gemini-3.8-flash`) via the @google/genai SDK, behind a ModelProvider interface. Function calling with JSON-schema tool declarations. Automatic function calling is disabled, so the server executes every call itself. Temperature 0.2, 25-second timeout. The model ID is configuration (`MODEL_ID`), not code.
Tools	Eight allowlisted tools (listed in section 03). None can confirm, approve, transfer money, fetch URLs or run code.
Data sources	User-entered records or CSV import (date, direction, amount, category, status, source reference). A fictional demo cooperative and eight fictional sandbox lender products, labelled `sandbox_fixture`. No empirical farm data or real loan terms are used.
Reference rates	Display only, never used in calculations. Default 1 USD = BDT 123.7054 (Bangladesh Bank reference rate, 19 Oct 2026); EUR derived with EUR/USD ≈ 1.1221 (BDT 138.8098). Users can edit both.
External services	GitHub Pages (web app), Render free web service (API), Neon Postgres free tier (database), Google Gemini API (model), GitHub Actions (CI). No real lender, payment, identity (KYC) or credit-bureau service is connected.

03 Agent Design

- Intent understanding, planning, tool use and state management

Intent understanding. The user writes in plain English, for example "When does my cash run short?", "Compare financing and stress-test the cheapest option" or "Prepare a sandbox application with the best option". Gemini receives the message, the recent conversation, a system prompt and the eight tool declarations, and decides which tools it needs. The system prompt is a behaviour guide only. It tells the model to use tools for every calculation, never to do money arithmetic itself, to quote formatted tool outputs, to treat record text as data, to ask for missing information, and to stop after preparing an action. Authorization never depends on the prompt.

Planning and tool use. Each message runs through a bounded loop:

1. The kill switch (`LIVE_AI_ENABLED`) and the per-session run quota (default 10 runs per hour) are checked.
2. Up to 3 model round trips per message. Before each call, a global daily provider-call budget (default 300) is reserved atomically in the database.
3. Each proposed call passes the allowlist, then strict zod argument validation, then runs through SandboxService with session-scoped authorization. The real result, or the real error, is returned to the model; failures are never replaced with invented output. At most 6 tool calls per message.
4. When `prepare_sandbox_submission` creates a pending action, the loop stops at once with status `awaiting_confirmation`. The model has no tool that can confirm.
5. The reply, run status and tool timeline are stored and returned to the browser, which shows them in the copilot panel.

Tool	What it does	Policy
get_business_records	Reads the workspace records; free-text descriptions are passed as `untrusted_description` data	allowed (read)
calculate_cashflow	Daily projection, first negative date, funding gap, low and critical alerts	allowed (read)
match_financing_options	Sizes and classifies every product, with reason codes and modeled costs	allowed (read)
run_stress_test	Holds one costed option fixed and replays four adverse scenarios	allowed (read)
create_application_draft	Saves a draft for an option that fits; submits nothing	allowed (sandbox write)
prepare_sandbox_submission	Freezes the exact payload, returns its stress check, creates a pending action	requires human confirmation
get_application_status	Reads the stored status of applications	allowed (read)
export_execution_log	Summarizes this session's recorded, redacted tool events	allowed (read)

In guided mode there is no model: explicit buttons in the copilot panel run the same tools deterministically, and the mode label says "Guided sandbox — no live AI".

State management. All live-mode state is held on the server per session, in Postgres: the workspace (versioned), cash records, the product catalog (versioned), drafts, pending actions, approvals, receipts, agent runs and tool events. Every tool call writes a tool event with the tool name, actor (user, model or guided), policy decision, outcome and record versions; these feed the run timeline and the evidence export. A pending action stores the canonical JSON payload, its SHA-256 hash, a nonce and a 5-minute expiry, and is bound to the draft, workspace and catalog versions.

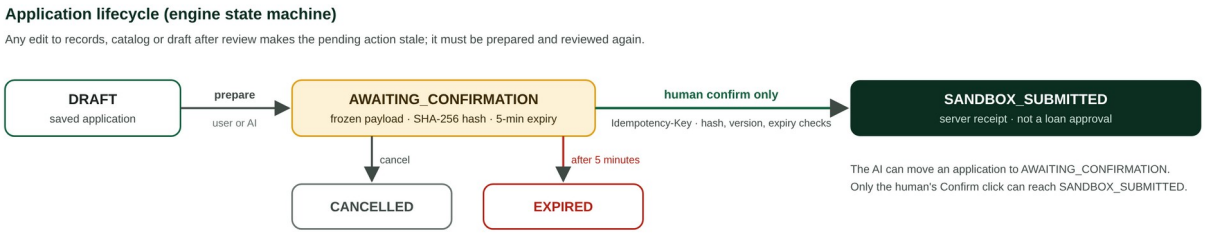


Figure 2. Application lifecycle. The AI can prepare; only the human can submit.

- Core algorithms and evaluation approach

Money and dates

Amounts are integer minor units: decimal strings at every boundary and `bigint` inside the engine, bounded to ±10^14, with integer round-half-up. Floating point is never used for money. Dates are civil `YYYY-MM-DD` strings with day-number arithmetic and no time-zone conversion.

Cash-flow projection

```
window       = [asOfDate, asOfDate + forecastDays − 1]
closing(d)   = opening(d) + inflows(d) − outflows(d)
funding_gap  = max(0, −min closing)
```

Cancelled events are excluded. Settled events before the window are already in the opening cash. Planned events dated before the window are listed as overdue and excluded, never silently counted. Days below the minimum buffer raise a low alert; negative days raise a critical alert.

Financing comparison

```
fee(P)       = fixed_fee + round_half_up(P × fee_bps / 10 000)
interest(P)  = round_half_up(P × rate_bps × days / (10 000 × 365))
net(P)       = P − fee(P)
```

1. Binary-search the minimum principal P within the product limits such that net(P) covers the required cash. This is exact because net(P) is monotonic.
2. Reject the product if the funds arrive after the first negative date.
3. Re-project cash with the net disbursement and the maturity repayment added; reject if cash goes negative again.
4. Report "needs information" if maturity falls beyond the forecast window or terms are missing.
5. Rank options that fit by lowest total modeled cost, then lowest repayment, then ID.

The financing target is either `gap` (cover the shortfall to zero) or `gap_plus_buffer` (also restore the minimum cash buffer). Products in another currency are never compared and get CURRENCY_MISMATCH.

Repayment stress test

The drafted loan (principal, disbursement date, maturity date) is held fixed while planned events are changed: harvest receipts 14 days late; sales −20%; costs +15%; and a combined case (10 days late, sales −10%, costs +10%). The horizon is extended to maturity. A scenario passes if

closing cash never goes below zero through maturity. The verdict is resilient, partly resilient or fragile. Scenarios are assumptions, not predictions.

Evaluation approach

- **Exact correctness.** Engine unit tests use exact expected values, for example proving that P−1 is insufficient, and that every demo product reaches its documented outcome.
- **Browser and server agree.** A parity test checks that API outputs are byte-identical (canonical JSON) to the engine for the same inputs.
- **Agent safety with a scripted model.** A mock provider proposes unknown tools, forged arguments, prompt-injected actions and confirmation attempts; the tests assert what was rejected, executed and logged.
- **End-to-end.** Playwright drives the built site on desktop and mobile through the full guided flow.
- **Live model.** One real Gemini run was inspected by hand: the reply figures matched the tool outputs and the agent stopped for human confirmation. This is one observed run, not a statistical evaluation; the live model is not called in CI, to avoid paid calls.

What evaluation changed. In that live run, the model prepared the cheapest option without mentioning that it failed all four stress scenarios. We fixed this in three places: `prepare_sandbox_submission` now returns a `stressCheck` with an explicit instruction to warn the user; the system prompt requires reporting it and suggesting "Gap + buffer"; and the review dialog shows the stress result and requires an extra acknowledgement when the terms are not resilient. A new API test checks the warning.

04 Security Design

- Tiered permissions and confirmation for sensitive actions

Tier	Can do	Cannot do	Enforced by
T0 Public visitor	Use guided mode in their own browser; read health; create an anonymous session (rate-limited)	Read any session's data	No data without a session; per-IP and global session quotas
T1 Session holder	Read and write their own workspace; analyse; draft; prepare; confirm their own pending actions; export evidence; delete the session	Access other sessions; bypass expiry or quotas	Bearer token (32 random bytes, only its SHA-256 hash stored); every query runs in a transaction scoped to the session; client-supplied IDs ignored
T2 Model (agent)	Call the 8 allowlisted tools inside a T1 session	Confirm, approve, transfer, fetch URLs, run code or SQL, act on another session, exceed 3 round trips or 6 tool calls	Allowlist, then zod validation, then the same service code as the UI; unknown tools rejected and logged; loop halts at a pending action
T3 Human confirmation	Move one pending action to SANDBOX_SUBMITTED	Confirm a stale, expired, cancelled or mismatched action	`POST /v1/actions/:id/confirm` (not in the tool registry); re-verification and atomic commit

Tier	Can do	Cannot do	Enforced by
T4 Operator	Set environment variables, kill switch and quotas	n/a	Render dashboard; secrets never in the repository or the web bundle

Confirmation of the sensitive action (sandbox submission) works in four steps:

1. **Prepare.** The exact payload (terms, dates, shared fields, record IDs) is frozen, hashed with SHA-256 and given a nonce and a 5-minute expiry.
2. **Review.** The dialog shows the exact terms, every field that will be shared, the record IDs, the hash prefix, a countdown and the stress-test result. Confirm stays disabled until the user ticks the review box, plus a second box if the terms are not resilient.
3. **Confirm.** The endpoint reloads the action and draft and re-checks session ownership, pending status, expiry, the draft, workspace and catalog versions, and the recomputed payload hash. It requires an `Idempotency-Key` and commits the submission, approval, audit event and receipt in one transaction under a row lock. The same key returns the same receipt; another key after submission gets 409.
4. **Receipt.** The receipt is labelled as a sandbox result: SANDBOX_SUBMITTED means only that the internal simulator received the application.

- Protection against prompt injection, privilege escalation and privacy leakage

Threat	Control	Test evidence
Prompt injection in record text	Descriptions are passed as `untrusted_description` data; the prompt says to treat them as data; the tool boundary holds even if the model obeys	“prompt injection inside record text does not cause unauthorized actions”
Model tries to confirm or escalate	No such tool exists; unknown tools rejected before execution; the loop halts at a pending action	“rejects unknown tools (including an attempted confirm)…”; “stops immediately after preparing…”
Malformed or malicious arguments	zod `.strict()` schemas and ID regexes	“rejects invalid tool arguments before execution”
Forged authority (role, approved flag, workspace ID)	Ignored; confirmation re-derives everything on the server	“requires an idempotency key and ignores forged approval flags”; “ignores client-supplied workspace ownership fields”
Cross-session access (IDOR)	Session-scoped transactions; unknown IDs return 404 without revealing existence	“another session cannot read or confirm…”
Change after review	Payload hash and versions bound; any edit makes the action stale	“a change after review makes the action stale”
Double click or replay	Idempotency-Key plus row lock (`SELECT … FOR UPDATE`)	Concurrency tests, in memory and on real PostgreSQL 16
Cost abuse of the model proxy	Kill switch; per-session and global daily quotas reserved before each call; per-IP rate limits; size limits; timeout	Quota and kill-switch tests verify no further provider call

Threat	Control	Test evidence
Secret leakage	Provider key only in the backend environment; no `VITE_*` secrets; CI secret-scans source and built bundle; health exposes no config	Health test; secret-scan step in CI
XSS and CSV formula injection	React renders text only; CSV export prefixes `= + - @` cells with an apostrophe	“stores HTML in descriptions as inert text”; CSV export test

Privacy. No passwords, identity documents, payment credentials or bank details are collected, and the UI asks users not to enter them. The application shares only the fields listed in the review dialog and record IDs, not record descriptions. Sessions expire after 120 minutes by default, and `DELETE /v1/session` deletes all of a session's rows. There are no analytics, trackers or cookies. Provider-side logs and database backups follow the providers' own retention policies.

Known risks (honest list)

- A bearer token is not identity: whoever holds it is that session. This is acceptable for a sandbox with no real-world effect, but it is not strong customer authentication or KYC.
- Guided-mode data and logs live in the browser and can be edited by the user; local receipts are labelled "browser only".
- Server logs are application records, not tamper-proof evidence. The evidence content hash is an integrity aid, not an attestation.
- Prompt-injection resistance is structural: the model may still say something misleading. Numbers in cards come from tools; free-text replies are model output and labelled as such.
- Quotas use fixed windows, so bursts at a window edge can briefly reach twice a limit; global caps bound the total.
- Free hosting can cold-start (about 50 seconds). There has been no third-party penetration test, no load test and no automated dependency-vulnerability gate in CI.

05 Deployment and Testing

- Environment, deployment instructions, test cases and known limitations

Environment

Part	Where	Configuration (names only; secrets live in the host dashboard)
Web app	GitHub Pages	Base path `/Farmlink-AI/`; `VITE_API_BASE_URL` (public API URL only); `VITE_BUILD_SHA`
API	Render free web service, Singapore, Node 22	Build `npm ci && npm run build -w backend`; start `node backend/dist/server.js`; health check `/v1/health`
Database	Neon Postgres	`DATABASE_URL` (secret); tables are created on startup
Model	Google Gemini API	`GEMINI_API_KEY` (secret), `MODEL_ID`, `LIVE_AI_ENABLED`

Part	Where	Configuration (names only; secrets live in the host dashboard)
Limits	API environment	`ALLOWED_ORIGINS` (exact origin), `SESSION_TTL_MINUTES=120`, `AGENT_RUNS_PER_SESSION_PER_HOUR=10`, `PROVIDER_CALLS_PER_DAY_GLOBAL=300`, `MAX_ROUND_TRIPS=3`, `MAX_TOOL_CALLS=6`

Run locally

1. Install Node.js 22 and Git.
2. `git clone https://github.com/Tanbin-CS/Farmlink-AI.git` and `cd Farmlink-AI`
3. `npm install` (CI uses `npm ci`)
4. `npm run dev:web`, then open http://localhost:5173 and click "Try the demo" (guided mode, no AI needed).
5. Optional live API: copy `backend/.env.example` to `backend/.env`, set the values, run `npm run dev:api`, then start the web app with `VITE_API_BASE_URL=http://localhost:8080`.
6. Checks: `npm run typecheck && npm run lint && npm test && npm run build`; browser tests with `npm run e2e -w frontend`.

Deploy

1. GitHub Pages: set Settings → Pages → Source to GitHub Actions. Every push to `main` runs `.github/workflows/pages.yml`, which typechecks, tests, builds, secret-scans and deploys.
2. Database: create a Neon project and copy its connection string into Render as `DATABASE_URL`.
3. API: create the Render service from `render.yaml` (Blueprint), add `GEMINI_API_KEY`, `MODEL_ID` and `DATABASE_URL` in the dashboard, and set `LIVE_AI_ENABLED=true`.
4. Verify: `/v1/health` reports `storage: postgres`, `durableStorage: true` and `aiAvailable: true`. The deployed API currently reports build `14753a3`.

Test cases and results

Results of commands actually run on 19–20 October 2026 (UTC). A test is listed as passed only if it ran.

Suite	Tests	Result
Engine unit tests (money, dates, cash flow, financing, currencies, stress test)	31	31 passed
Frontend unit tests (guided client)	5	5 passed
API safety tests (in-memory store, scripted model)	27 + 2 opt-in	27 passed; the 2 Postgres tests need `TEST_DATABASE_URL` and passed against real PostgreSQL 16 on earlier commit e715f2a
Browser end-to-end (Playwright, desktop and mobile)	12	11 passed, 1 skipped on mobile by design
Typecheck, lint, build, secret scan	—	passed
GitHub Actions CI, commit 14753a3	—	success

Suite	Tests	Result
Live Gemini end-to-end (manual)	1 run	passed (see execution evidence)

Key test case	Expected	Observed
Hidden instruction inside a record	No unauthorized action	`approve_loan` rejected; invalid arguments rejected; no application created
Model proposes an unknown tool or tries to confirm	Rejected and logged	Both rejected; nothing submitted
Another session requests a record or confirms an action	Denied without leaking existence	404 with no ID echoed
Confirm without key, or with forged approval flags	Refused	400; status stays AWAITING_CONFIRMATION
Records edited after review	Action becomes stale	409 STALE_DRAFT; rejection logged
Expired action or session	Denied	409 ACTION_EXPIRED; 401 SESSION_EXPIRED
Three parallel confirms with the same key	Exactly one submission	One receipt returned three times; a different key gets 409
Model outage or exhausted quota	Honest error, no further provider call	503 PROVIDER_ERROR or 429; run stored as failed; provider call count verified
Preparing a fragile option	Model told to warn; UI warns	`stressCheck` verdict fragile, 0 of 4; review dialog warning and second checkbox (e2e)

Execution evidence

- `docs/evidence/2026-10-19-guided-local-run.json`: an automated guided-mode run (no model): demo, draft, prepare, human confirm, export.
- `docs/evidence/2026-10-19-live-cloud-gemini-run.json`: a real run on the public site against the Render API (backend commit 6eb18c4), `gemini-3.8-flash`, 19 Oct 2026 22:10–22:13 UTC. The model called calculate_cashflow, match_financing_options, run_stress_test, create_application_draft and prepare_sandbox_submission, then stopped at awaiting_confirmation. The human confirmed in the review dialog, producing receipt `rcpt_dlFmbi2JgE7l`. That run used in-memory storage; the API now uses Neon Postgres.
- Each file is an unedited export from Activity → Export evidence bundle, with run IDs, timestamps, tool events, policy decisions and receipts. Its content hash is an integrity aid.

Known limitations

- Cash-based daily projection only: no accruals, inventory or profit; same-day flows are netted.
- Only simple-interest, bullet-repayment loans with an optional upfront fee are modeled; reducing-balance, compounding and instalment products are marked unsupported.

- "Fits" means fits the stated sandbox rules: no creditworthiness assessment, default probability, yield or price forecast is produced.
- Lenders are simulated. There is no KYC, credit bureau, government API, payment or disbursement.
- Sessions are anonymous and expire after 2 hours; they are not user accounts. The interface is in English; Bengali is future work.
- Live mode depends on free hosting (cold starts of about 50 seconds) and on the team's Gemini quota.